K.R. Mangalam University

DSA Assignment:3

Submitted by:

Madhav kalra
2501730284
B.TECH CSE(AI&ML)
Section: A

Submitted to:

Mrs. Neetu Chauhan

Analysis and Comparison

1. A table of recorded execution times (9 experiments × 3 algorithms = 27 results).

Running Experiments...

Size	Type	Insertion	Merge	Quick
1000	random	40.57	2.62	2.62
1000	sorted	0.31	3.16	2.02
1000	reverse	65.83	3.18	2.31
3000	random	507.88	8.89	3.97
3000	sorted	0.32	10.06	4.56
3000	reverse	1035.69	10.53	7.78
5000	random	1476.65	25.43	6.80
5000	sorted	0.55	15.97	9.98
5000	reverse	2738.38	17.83	13.78

2. A short comparison paragraph for each input type:

- **Random Input :**Insertion sort is slower due to many comparisons.
  Merge and quick sort perform faster using divide and conquer.
- Sorted Input :Insertion sort becomes faster because fewer shifts are needed.
  Quick sort may slow down if pivot is not chosen properly.
  Merge sort remains consistent.
- **Reverse Sorted Input:**Insertion sort performs worst due to maximum shifts.

Quick sort may also perform poorly depending on pivot. Merge sort remains efficient.

3. How does $O(n^2)$ vs O(nlogn) match your timing results:

Insertion sort has $O(n^2)$ complexity, so it becomes slow for large inputs.

Merge sort and quick sort have O(n log n) complexity, so they perform better for large data.

Experimental results show:

- Insertion sort time increases rapidly
- Merge and quick sort grow slowly

4. Special note for Quick Sort:

• Mention worst-case O(n2) when pivot is poor (e.g., sorted input with last element pivot).

Quick sort is usually fast (O(n log n)), but in worst case it becomes $O(n^2)$.
This happens when pivot is not chosen properly, like taking last element in an already sorted list.
In that case, partitions are unbalanced and sorting becomes slow.

• If you observe slow performance for sorted or reverse-sorted, explain why

If performance is slow for sorted or reverse-sorted input, it is because quick sort is using a poor pivot (like last element).

This creates unbalanced partitions, so the algorithm behaves like $O(n^2)$ and takes more time.

Conclusion:

- Insertion sort is used for small or nearly sorted data.
- Merge sort is reliable and efficient for all the cases.
- Quick sort is very fast on random data but depends on pivot selection.